

Code:

```
from tkinter import *
from tkinter import ttk, messagebox

class CallWindow:
    def __init__(self, win):
        win.title("Long Distance Charge Calculator")
        win.geometry("600x450")
        win.configure(bg="#ffffff")

        # Make window responsive
        for i in range(6):
            win.rowconfigure(i, weight=1)
        for j in range(4):
            win.columnconfigure(j, weight=1)

        # ===== RATES TABLE =====
        self.rates = {
            "American Region": {"Day": (50, 3), "Night": (45, 3)},
            "Asian Region": {"Day": (30, 2), "Night": (27, 2)},
            "African Region": {"Day": (40, 3), "Night": (36, 3)},
            "European Region": {"Day": (35, 2), "Night": (30, 2)},
        }

        label_font = ("Consolas", 12, "bold")
        entry_font = ("Consolas", 12)
        button_font = ("Consolas", 11, "bold")

        # ===== LABELS =====
        self.lbl_duration = Label(win, text="Length of Call (minutes):",
            fg="#1e1e1e", bg="#ffffff", font=label_font)
        self.lbl_dest = Label(win, text="Destination Code:", fg="#1e1e1e",
            bg="#ffffff", font=label_font)
        self.lbl_time = Label(win, text="Time Code:", fg="#1e1e1e",
            bg="#ffffff", font=label_font)
        self.lbl_summary = Label(win, text="Transaction Summary:",
            fg="#1e1e1e", bg="#ffffff", font=label_font)

        # ===== ENTRY =====
        self.duration_entry = Entry(win, bd=3, font=entry_font, bg="#ffffff",
            fg="#1e1e1e", insertbackground="#1e1e1e")

        # ===== COMBOBOX =====
        self.dest_var = StringVar()
        self.dest_combo = ttk.Combobox(win, textvariable=self.dest_var,
            state="readonly", font=entry_font, values=list(self.rates.keys()))
        self.dest_combo.configure(width=18)
```

```

# ===== RADIO BUTTONS =====
self.time_var = StringVar()
self.rb_day = Radiobutton(win, text="Day Time",
variable=self.time_var, value="Day", fg="#1e1e1e", bg="#ffffff",
selectcolor="#ffffff", font=entry_font)
self.rb_night = Radiobutton(win, text="Night Time",
variable=self.time_var, value="Night", fg="#1e1e1e", bg="#ffffff",
selectcolor="#ffffff", font=entry_font)

# ===== OUTPUT TEXTBOX =====
self.output_box = Text(win, width=50, height=8, bd=3, bg="#2e2e2e",
fg="#00ffea", font=("Consolas", 12), insertbackground="#00ffea")

# ===== BUTTONS =====
self.btn_compute = Button(win, text="Compute Charge",
command=self.compute, bg="#00ffea", fg="#1e1e1e", font=button_font)
self.btn_reset = Button(win, text="Reset", command=self.reset,
bg="#00ffea", fg="#1e1e1e", font=button_font)
self.btn_about = Button(win, text="About", command=self.about,
bg="#00ffea", fg="#1e1e1e", font=button_font)
self.btn_close = Button(win, text="Close", command=win.destroy,
bg="#ff4d4d", fg="#1e1e1e", font=button_font)

# ===== GRID PLACEMENT =====
self.lbl_duration.grid(row=0, column=0, sticky="w", padx=20, pady=5)
self.duration_entry.grid(row=0, column=1, sticky="ew", padx=20,
pady=5, columnspan=3)

self.lbl_dest.grid(row=1, column=0, sticky="w", padx=20, pady=5)
self.dest_combo.grid(row=1, column=1, sticky="ew", padx=20, pady=5,
columnspan=3)

self.lbl_time.grid(row=2, column=0, sticky="w", padx=20, pady=5)
self.rb_day.grid(row=2, column=1, sticky="w", padx=5, pady=5)
self.rb_night.grid(row=2, column=2, sticky="w", padx=5, pady=5)

self.lbl_summary.grid(row=3, column=0, sticky="w", padx=20, pady=5)
self.output_box.grid(row=4, column=0, columnspan=4, sticky="nsew",
padx=20, pady=5)

self.btn_compute.grid(row=5, column=0, sticky="ew", padx=20, pady=10)
self.btn_reset.grid(row=5, column=1, sticky="ew", padx=20, pady=10)
self.btn_about.grid(row=5, column=2, sticky="ew", padx=20, pady=10)
self.btn_close.grid(row=5, column=3, sticky="ew", padx=20, pady=10)

# ===== FUNCTIONS =====
def compute(self):
    self.output_box.delete("1.0", END)

    try:
        minutes = int(self.duration_entry.get())
        if minutes <= 0:
            raise ValueError
    except ValueError:
        messagebox.showerror("Invalid Input", "Duration must be a
positive integer.")
    return

```

```

        destination = self.dest_var.get()
        timecode = self.time_var.get()

        if not destination or not timecode:
            messagebox.showerror("Missing Input", "Please select destination
and time code.")
            return

        rate, per_min = self.rates[destination][timecode]
        total_cost = (minutes / per_min) * rate

        summary = (
            f"Duration of Call:      {minutes} minutes\n"
            f"Destination:                {destination}\n"
            f"Time Code:                    {timecode}\n"
            f"Rate:                          ₱{rate} every {per_min} minutes\n"
            f"-----\n"
            f"TOTAL CHARGE:                  ₱{total_cost:.2f}"
        )

        self.output_box.insert(END, summary)

    def reset(self):
        self.duration_entry.delete(0, END)
        self.dest_var.set("")
        self.time_var.set("")
        self.output_box.delete("1.0", END)

    def about(self):
        messagebox.showinfo("About", "Hello! dear")

# ===== RUN WINDOW =====
if __name__ == "__main__":
    window = Tk()
    app = CallWindow(window)
    window.mainloop()

```

Output:

Long Distance Charge Calculator

Length of Call (minutes):

60

Destination Code:

Asian Region

Time Code:

☒ Day Time ☐ Night Time

Transaction Summary:

Duration of Call: 60 minutes

Destination: Asian Region

Time Code: Day

Rate: ₱30 every 2 minutes

TOTAL CHARGE: ₱900.00

Compute Charge

Reset

About

Close